

Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time)

Disciplina DEC107 — Processamento Paralelo

Semestre 2026-2 — Agosto de 2026

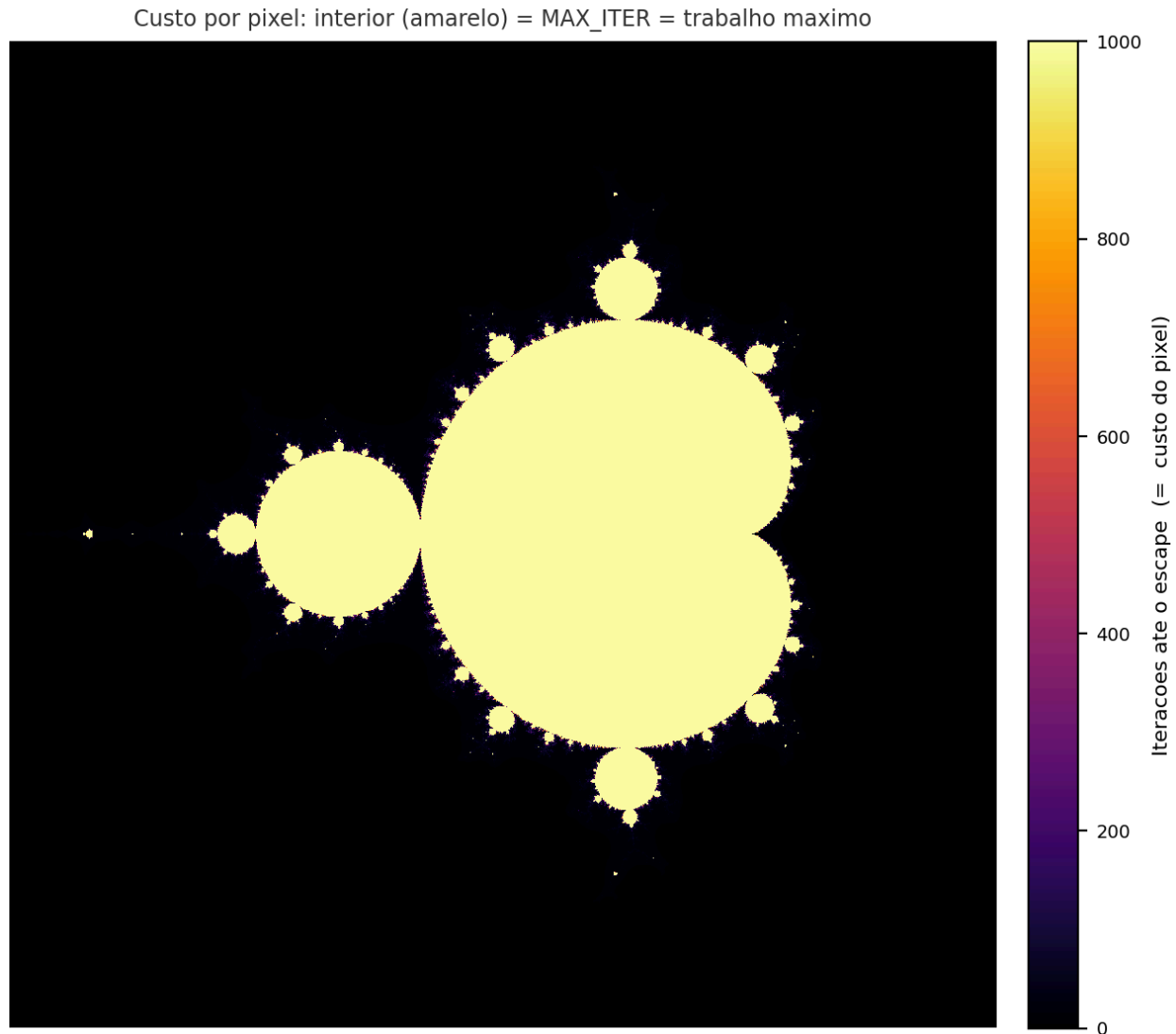
1. Contextualização e Motivação

Este projeto tem como objetivo principal avaliar o aprendizado dos conceitos de processamento paralelo da disciplina DEC107. Propomos a geração do conjunto de Mandelbrot pelo método de *escape-time* (tempo de escape) como problema central. Trata-se de um benchmark excelente por reunir três propriedades didáticas raras num mesmo problema: é embaraçosamente paralelo em sua estrutura básica, é fortemente **desbalanceado** em custo por elemento, e é determinístico — o que torna a validação de corretude simples e objetiva. A implementação de todas as etapas será realizada na linguagem C.

2. O que é o Conjunto de Mandelbrot?

O conjunto de Mandelbrot é o conjunto dos números complexos c para os quais a sequência definida por $z_0 = 0$ e $z_{n+1} = z_n^2 + c$ permanece limitada. Na prática, para cada ponto c do plano complexo, itera-se essa recorrência e conta-se quantas iterações são necessárias até que $|z|$ ultrapasse um raio de escape (tipicamente 2). Se o ponto não escapar dentro de um número máximo de iterações, considera-se que ele pertence ao conjunto.

Esse número de iterações até o escape — o *escape-time* — é usado para colorir cada pixel da imagem, produzindo a característica fronteira fractal do conjunto. O ponto central para este projeto é que **o custo computacional de cada pixel é proporcional ao seu escape-time**: pixels fora do conjunto tendem a escapar em poucas iterações, enquanto pixels dentro do conjunto (ou próximos da fronteira) consomem todas as iterações possíveis. Essa heterogeneidade de custo é a fonte natural de desbalanceamento de carga que o projeto explora.



3. Objetivo Geral

Desenvolver e otimizar um algoritmo de geração do conjunto de Mandelbrot por escape-time, aplicando técnicas de paralelismo em memória compartilhada (OpenMP), memória distribuída (MPI) e GPUs (CUDA), com ênfase na análise de balanceamento de carga e de escalabilidade.

4. Objetivos Específicos

Ao final do projeto, as duplas deverão ser capazes de:

1. Implementar uma versão serial e versões paralelas do gerador de Mandelbrot em C.
2. Aplicar e analisar o desempenho de paralelismo em OpenMP, MPI e CUDA.
3. Medir e interpretar métricas de desempenho como tempo de execução, Speedup e Eficiência.

4. Diagnosticar e mitigar o desbalanceamento de carga inerente ao problema, escolhendo estratégias de distribuição de trabalho apropriadas a cada paradigma.
5. Comparar o desempenho e a escalabilidade das diferentes abordagens paralelas, relacionando os resultados às leis de Amdahl e Gustafson.
6. Apresentar uma análise crítica dos resultados obtidos em relatórios e apresentações orais.

5. Descrição do Benchmark

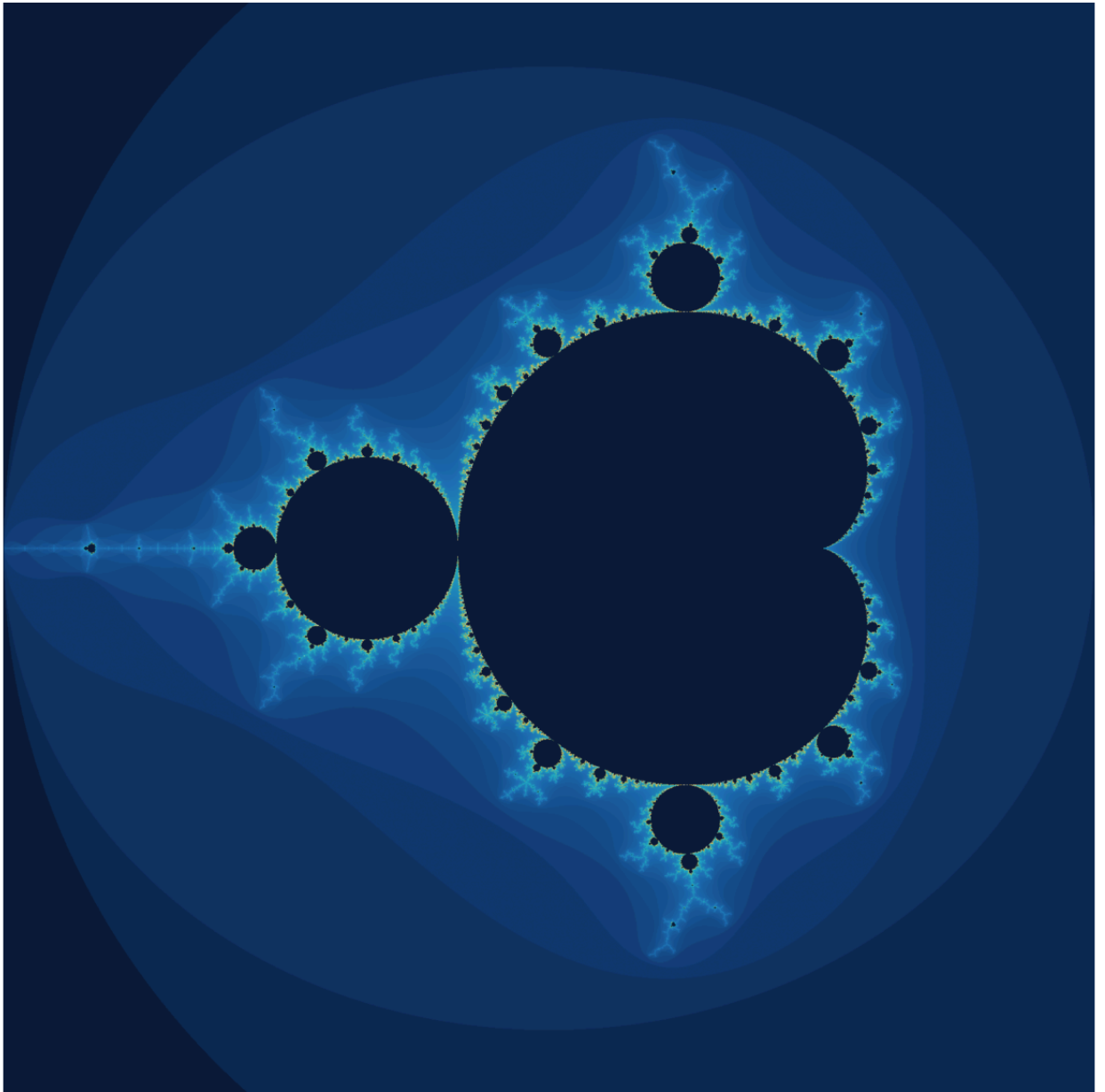
O benchmark consiste em calcular, para cada pixel de uma imagem que representa uma região do plano complexo, o escape-time da recorrência de Mandelbrot. A saída é uma matriz de contagens de iteração (uma por pixel), a partir da qual se gera a imagem colorida. Será fornecida uma versão serial de referência para validação da corretude.

5.1. Especificação do cálculo. Para o pixel de coordenadas de imagem (px, py), determina-se o ponto complexo c correspondente por mapeamento linear da região do plano definida abaixo. Itera-se $z_{n+1} = z_n^2 + c$ a partir de $z_0 = 0$, com critério de escape $|z|^2 > 4$ (equivalente a $|z| > 2$, evitando a raiz quadrada). A contagem de iterações até o escape, limitada por `MAX_ITER`, é o valor do pixel. Toda a aritmética deve usar ponto flutuante de dupla precisão (`double`).

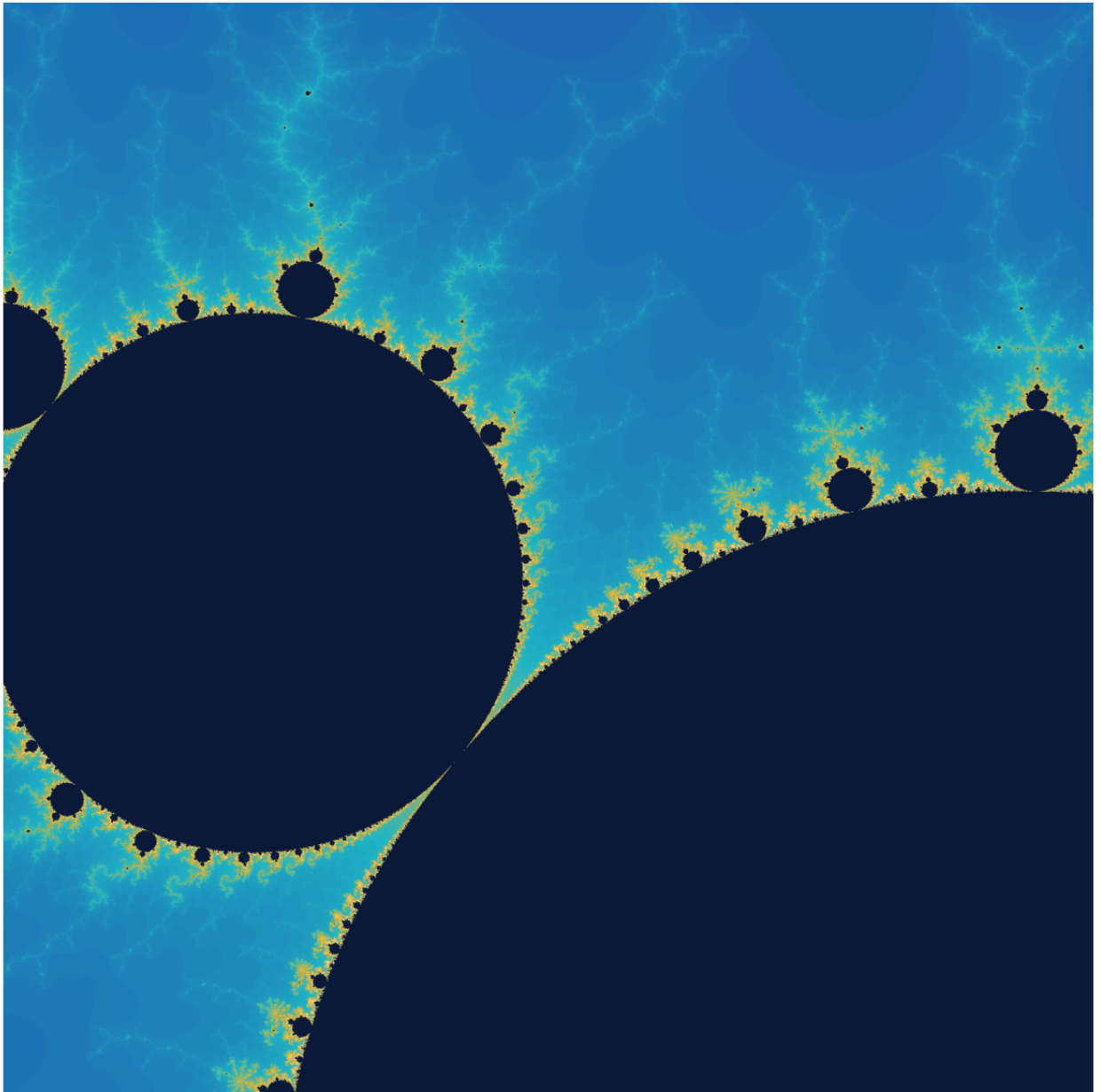
5.2. Input padrão (obrigatório). Para garantir a comparabilidade do ranking entre duplas, os seguintes parâmetros são fixos e devem ser usados em todos os experimentos oficiais:

- **Região do plano complexo (vista completa):** $\text{Re} \in [-2, 0, 1, 0]$, $\text{Im} \in [-1, 5, 1, 5]$.
- **Resolução base:** 4096×4096 pixels.
- **Número máximo de iterações:** `MAX_ITER = 1000`.
- **Raio de escape:** $|z| = 2$ (teste por $|z|^2 > 4$).
- **Precisão:** `double`.

Conjunto de Mandelbrot - vista completa (Re in $[-2,1]$, Im in $[-1.5,1.5]$)



Zoom: região de espirais (detalhe da fronteira fractal)

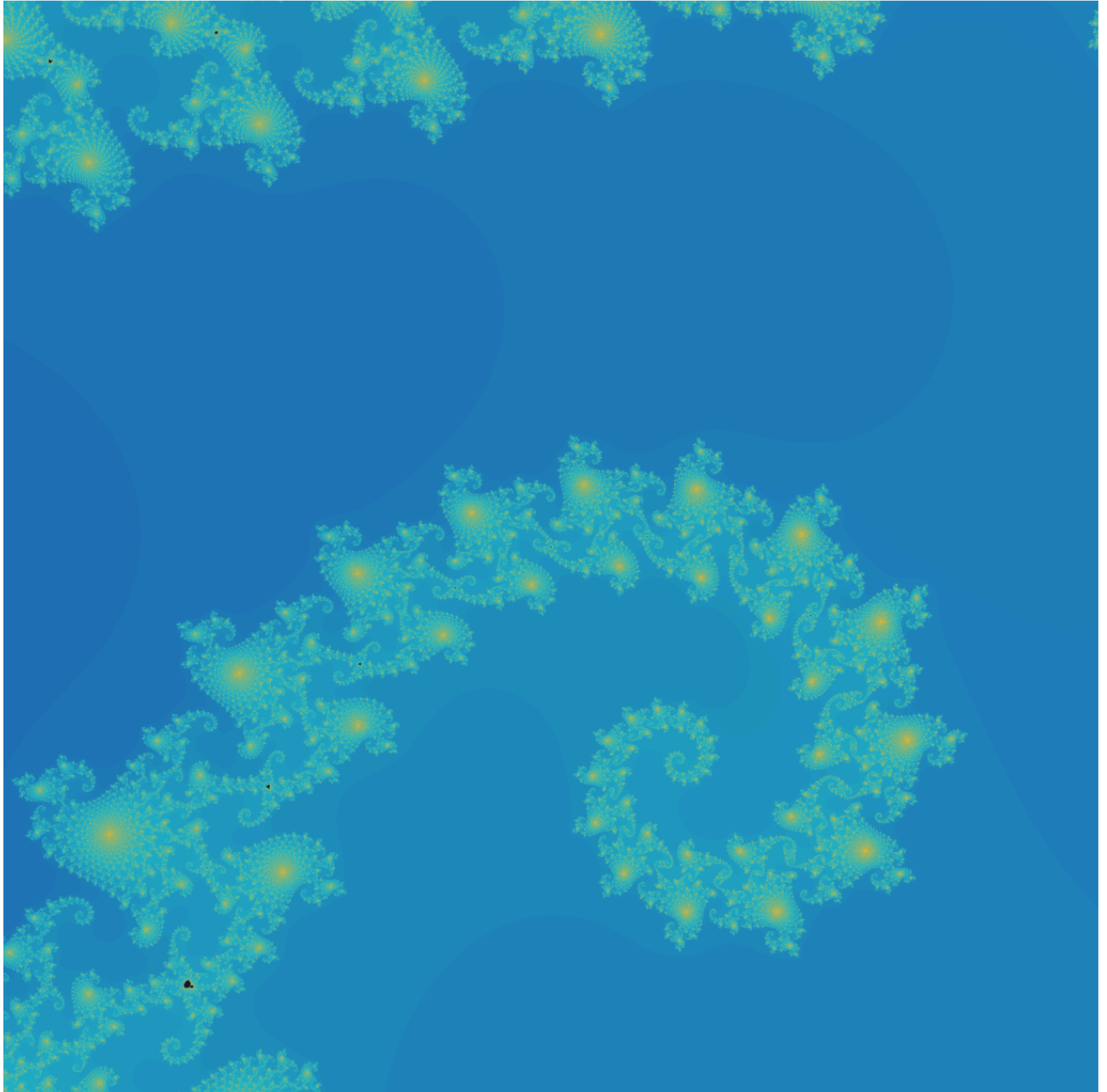


5.3. Casos de estresse (para as análises de escalabilidade). Além do caso base, os seguintes casos devem ser usados nas análises indicadas:

- **Escalabilidade forte (strong scaling):** manter a resolução base (4096×4096) e variar o número de threads/processos.
- **Escalabilidade fraca (weak scaling):** manter o trabalho por unidade de processamento aproximadamente constante, escalando a resolução (por exemplo 4096^2 , 8192^2 , 16384^2) proporcionalmente aos recursos.
- **Caso de desbalanceamento acentuado (obrigatório na análise de carga):** região de zoom no "vale dos cavalos-marinhos", centrada em $(-0,743643887, 0,131825904)$ com

largura $3,0 \times 10^{-3}$ no eixo real (mantendo proporção da imagem), e **MAX_ITER = 5000**. Essa região concentra grande quantidade de pixels de custo máximo e evidencia problemas de distribuição de trabalho.

Zoom: vale dos cavalos-marinhos (centro $-0.743643887 + 0.131825904i$, MAX_ITER=5000)



5.4. Formato de saída. A saída canônica é a matriz de contagens de iteração gravada em arquivo binário (inteiros de 32 bits, ordem row-major), acompanhada de uma imagem em formato PGM ou PPM para inspeção visual. A imagem colorida serve à conferência humana; a validação de corretude é feita sobre a matriz de contagens.

5.5. Corretude. A saída paralela deve reproduzir a matriz de contagens da versão serial de referência. Por se tratar de um problema determinístico com contagens inteiras, a expectativa é de igualdade exata. Admite-se, no máximo, divergência em uma fração ínfima de pixels de fronteira (até 0,01% dos pixels, cada um diferindo em no máximo 1 iteração), desde que atribuível a reassociação de operações de ponto flutuante e explicitamente justificada no relatório. Divergências acima desse limite indicam erro de implementação e invalidam a pontuação de desempenho.

6. Regras do Projeto

1. O trabalho será realizado em duplas.
2. A submissão do código-fonte e dos relatórios será feita via GitHub.
3. Cada etapa do projeto exige a entrega de código-fonte, relatório técnico e resultados experimentais.
4. Os experimentos das etapas OpenMP e MPI devem ser realizados em computadores pessoais.
5. A etapa CUDA poderá exigir o uso de servidores remotos com hardware específico (GPU).
6. A corretude da implementação é obrigatória: a saída paralela deve ser equivalente à saída da versão serial de referência, conforme o critério da Seção 5.5.
7. O uso e a análise das métricas de desempenho são mandatórios em todas as etapas.
8. Os experimentos oficiais devem usar o input padrão da Seção 5.2; os casos de estresse da Seção 5.3 devem ser usados nas análises correspondentes.
9. Os relatórios devem registrar detalhadamente as condições do ambiente de execução (hardware, sistema operacional, compilador e parâmetros).

7. Cronograma das Três Etapas

7.1. Etapa 1 — OpenMP

Prazo: 2 semanas após a aula teórica.

Foco: Paralelismo em memória compartilhada. Distribuição do trabalho entre threads e, sobretudo, tratamento do desbalanceamento de carga por meio das políticas de escalonamento.

Entrega: Código serial e OpenMP, relatório da etapa.

Análise Principal no Relatório: Impacto do número de threads no desempenho; comparação entre as políticas de escalonamento (**static**, **dynamic** e **guided**) e o efeito do tamanho de *chunk*; Speedup, Eficiência e discussão fundamentada sobre balanceamento de carga, usando o caso de desbalanceamento acentuado da Seção 5.3.

7.2. Etapa 2 — MPI

Prazo: 2 semanas após a aula teórica.

Foco: Paralelismo em memória distribuída. Decomposição do domínio da imagem entre processos, estratégias de distribuição e custo de coleta do resultado.

Entrega: Código MPI, relatório da etapa.

Análise Principal no Relatório: Comparação entre pelo menos duas estratégias de distribuição — por exemplo, decomposição estática por blocos/faixas contíguas versus distribuição cíclica de linhas ou esquema mestre-trabalhador (*master-worker*) com balanceamento dinâmico; custo de comunicação (coleta da imagem via **Gather/Gatherv** e, quando aplicável, sobrecarga do mestre); Speedup, Eficiência, Fator de Balanceamento de Carga e comparação com a solução OpenMP.

7.3. Etapa 3 — CUDA

Prazo: 2 semanas após a aula teórica.

Foco: Paralelismo massivo em GPUs. Desenvolvimento do kernel de escape-time, configuração de blocos e grades de threads e uso eficiente da memória da GPU.

Entrega: Código CUDA, relatório final da etapa.

Análise Principal no Relatório: Desempenho do kernel; configuração de blocos/grade e sua influência no desempenho; discussão sobre **divergência de warps** decorrente do escape-time variável e eventuais estratégias de mitigação; custo de transferência host-device; Speedup, Eficiência e análise de ganhos e limitações em comparação com as versões anteriores.

8. Critérios de Avaliação

A avaliação será baseada nos seguintes critérios:

1. Corretude: A implementação deve produzir resultados corretos, equivalentes à versão serial (Seção 5.5).
2. Desempenho: A solução deve demonstrar ganhos significativos de tempo de execução em relação à versão serial.
3. Qualidade do Paralelismo: Uso adequado e otimizado das tecnologias (OpenMP, MPI, CUDA), incluindo o tratamento explícito do desbalanceamento de carga.
4. Análise dos Resultados: Capacidade da dupla de interpretar e discutir o comportamento da aplicação, os impactos do paralelismo e a escalabilidade (Amdahl/Gustafson).
5. Qualidade do Relatório e Apresentação Oral: Clareza, organização e correção técnica da documentação e da apresentação.

9. Métricas Obrigatórias

As seguintes métricas devem ser apresentadas e analisadas em todas as etapas:

- **Tempo de Execução:** O tempo total gasto para gerar a imagem (excluindo, quando pertinente, o tempo de escrita em disco, que deve ser reportado separadamente).
- **Speedup:** A razão entre o tempo de execução sequencial e o tempo de execução paralelo. Indica o ganho de velocidade.
- **Eficiência:** O Speedup dividido pelo número de processadores/threads. Indica o quão bem os recursos paralelos estão sendo utilizados.
- **Fator de Balanceamento de Carga:** Distribuição efetiva do trabalho entre threads/processos (por exemplo, razão entre o tempo do elemento mais carregado e o tempo médio, ou entre carga máxima e mínima). Esta métrica é central neste problema e deve ser analisada com o caso de desbalanceamento acentuado.
- **Discussão sobre Escalabilidade:** Análise de escalabilidade forte e fraca, relacionando os resultados às leis de Amdahl e Gustafson.

10. Sistema de Pontuação para Competição entre Duplas

Para incentivar a excelência e a análise crítica, será adotado um sistema de pontuação competitivo:

- A correteza da solução é um pré-requisito fundamental. Implementações incorretas não serão pontuadas em desempenho.
- Será estabelecido um ranking de desempenho para cada etapa, considerando o Speedup relativo obtido pela dupla em seu próprio hardware, sobre o input padrão da Seção 5.2. As duplas com os melhores desempenhos relativos receberão pontuação extra.
- Bônus serão concedidos por:
 - Análise técnica aprofundada e bem fundamentada nos relatórios, em especial no diagnóstico e na mitigação do desbalanceamento de carga.
 - Qualidade excepcional da apresentação oral.
 - Identificação e implementação de otimizações criativas (por exemplo, distribuição dinâmica de trabalho, redução de divergência em GPU, ou exploração de simetria do conjunto).

11. Orientações para os Relatórios

Cada relatório deve ser conciso e focar nos resultados da etapa. Deve incluir:

- **Introdução:** Breve descrição do objetivo da etapa.
- **Metodologia:** Detalhes da implementação paralela e da estratégia utilizada, incluindo a estratégia de distribuição de trabalho.
- **Ambiente de Execução:** Registro de hardware, sistema operacional, compilador e parâmetros de execução.

- Resultados: Apresentação das métricas obrigatórias (tempos, Speedup, Eficiência, balanceamento de carga) em gráficos claros.
- Análise: Discussão dos resultados, comparação com a versão serial (e com etapas anteriores, quando aplicável), identificação de gargalos e oportunidades de otimização, com atenção ao desbalanceamento e à escalabilidade.
- Conclusão: Resumo dos principais aprendizados e desafios da etapa.

12. Considerações Finais

Este projeto oferece uma oportunidade única para aplicar e aprofundar os conhecimentos em processamento paralelo em um contexto prático e visualmente rico. Ao contrário de um problema regularmente balanceado, a geração do conjunto de Mandelbrot expõe as duplas ao desafio central do HPC de distribuir trabalho heterogêneo de forma eficiente — tema que se manifesta de maneira distinta em cada um dos três paradigmas. A competição saudável entre as duplas visa estimular a busca por soluções eficientes e aprimorar a capacidade de análise crítica. O foco é no aprendizado e na compreensão dos desafios e benefícios do paralelismo em HPC.